Implementation Plan: Phase 1 – Development Environment & Project Setup

System specifications and architectural roadmap for Phase 1 Setup of the PERN/PostgreSQL Helpdesk System.

1. Phase Objective

Execute the project initialization steps to establish a running, connected full-stack PERN application. Install client and server package dependencies, initialize Prisma Client and map connectivity to PostgreSQL, bind security and traffic logging middleware in the Express application, and configure Axios client wrappers and routes in React. Create a diagnostics endpoint to verify client-to-database integration.

2. Features Covered

- **Project Skeleton Configurations**: Initializing package files and runtime commands in client and server folders.
- **Prisma ORM Initialization**: Instantiating the Prisma Client and connecting it to a local or remote PostgreSQL database instance.
- **Express Server Pipelines**: Setting up security (Helmet), cross-origin policies (CORS), request limits (Express Rate Limiter), and traffic logger (Morgan).
- **API Diagnostics Routing**: Deploying versioned health router endpoints.
- **Winston/Pino logger setup**: Structuring system runtime logging files.
- **Client App scaffolding**: Mounting Tailwind CSS configuration rules, instantiating standard Axios clients, and mapping React routes.

3. Functional Requirements

- **Health Telemetry**: The server must expose a public path checking server uptime, timestamp data, and database connection state.

- **Client Diagnostics Panel**: The frontend client must show system health statuses, transitioning panels dynamically from loading states to active states based on API response metrics.

4. Non-Functional Requirements

- **Latencies Limit**: The server response latency for health checks must be under 50ms.
- **Log Rotation**: Automatically manage and rotate log files in storage directories to protect disk allocations.
- **Header Shielding**: Conceal server versions and signatures in response headers using Helmet policies.
- **Rate Limits**: Prevent denial-of-service attempts by setting maximum connection parameters per IP address.

5. PostgreSQL Database Design

No business database tables or models are created in this phase. The Health Check endpoint will verify PostgreSQL connectivity directly through the Prisma Client. Business entities such as Users, Roles, Tickets, Categories, Priorities, Comments, Attachments, Knowledge Base, Activity Logs, Notifications, and AI-related entities will be introduced in later phases.

6. Prisma Schema Changes (Conceptual)

The health check logic will verify connectivity directly through database check functions exposed by the Prisma Client. No Prisma schemas or models are declared.

7. Table Relationships

No complex model relationships are instantiated in this phase.

8. Foreign Key Relationships

No foreign key mappings exist in this phase.

9. Indexing Strategy

No tables are created in this phase, so no custom indices are configured.

10. Folder Structure Changes

Establish the detailed MERN/PERN folder structure layout: Server: controllers, config, routes, middleware, utils, and validators directories. Client: src/components, pages, services, layouts, routes, styles, and utils folders.

11. Models Required

None. Business entities are introduced in later phases.

12. Controllers Required

`healthController.js` : Extracts system metrics and queries PostgreSQL database status via Prisma Client operations.

13. Services Required

`prismaClient.js` (Database service wrapper): Establishes a single cached instance of `PrismaClient` to handle connections and query logging logs.

14. Routes / API Endpoints

All API paths follow versioning guidelines. Diagnostics Route: /api/v1/health (Method: GET, Auth: Public, Headers: Accept: application/json).

15. Middleware Required

- **Helmet**: Customizes HTTP response header values.
- **CORS**: Restricts access to configured origins.
- **Morgan**: Records incoming traffic requests.
- **Express Rate Limiter**: Limits public calls using IP addresses.
- **Central Error Handler**: Catches operational exceptions and yields standard JSON configurations.

16. Frontend Pages Required

`HealthDashboard` : Main view container rendering diagnostic status cards.

17. React Components Required

`Loader` : Reusable loader spinner. `DiagnosticsCard` : Displays components health indicators.

18. Business Logic Flow

React client loads → Trigger Axios call to backend → Express captures request → Queries database via Prisma Client → Prisma executes diagnostics query on PostgreSQL → Connection verified → Server returns metadata → Client updates states.

19. AI Integration Flow

No active AI processes are executed in this phase. Environmental variables must include key placeholders for the Gemini API.

20. Security Considerations

- **Credentials Isolation**: Ensure `.env` is listed inside `.gitignore` to prevent leaking connection secrets.
- **CORS Access Scope**: Lock origin paths to the client app dev domain.
- **Header Protections**: Set Helmet policies to hide signatures.

21. Validation Rules

The server startup logic must check for the presence of `PORT` and `DATABASE_URL` configurations. Terminate execution if variables are missing.

22. Error Handling Strategy

Catch routing errors via Express fallthrough handlers. Map Prisma connection errors to operational validation objects. Mask internal database traces from payload outputs.

23. Logging Requirements

Integrate structured Winston logs. Save error traces to error logs and request metrics to combined logs.

24. Performance Optimization

Connection Re-use: Cache a single static client instance of the database mapper to prevent pool starvation. Connection Pooling: Set pool allocation thresholds within the connection string query parameters.

25. Transaction Requirements

No database transactions are run in this phase.

26. Edge Cases

PostgreSQL Database Offline: Ensure the Express backend starts successfully even if PostgreSQL is offline on boot, returning a database-disconnected error status in the health response.

27. Testing Checklist

- Verify backend runs on port.
- Verify database connection is tracked by checking health check endpoint when PostgreSQL is active.
- Verify database disconnected state resolves to a health check error response when PostgreSQL is offline.
- Verify React UI mounts and fetches status values via Axios.

28. API Response Contracts

Standardized JSON outputs for success and failure conditions. Success: contains server metrics, database state, server uptime, and timestamp values. Failure: contains description and components statuses.

29. Postman Testing Plan

Create request targeting the health endpoint. Confirm returned HTTP status code is 200 and check response data layout.

30. Git Commit Message

Configure Git commit standard rules to track setup actions using structural scope descriptions.

31. Deliverables

- Client and Server configurations (`package.json` , `tailwind.config.js`).
- Prisma schema setup files (`prisma/schema.prisma`).
- Database client wrapper configurations.
- Express app pipelines.
- Health check router and controller systems.
- Visual diagnostics screens.

32. Acceptance Criteria

- Prisma Client successfully executes connectivity queries against the PostgreSQL database.
- Diagnostics page queries the health check API and updates indicator card elements based on server-database health telemetry.
- Development server executes without error and logs database connectivity events.

33. Dependencies on Previous Phases

Phase 0 – Project Initialization & Repository Setup: The directory layout must be initialized and Git remote origins mapped.

34. Preparation for the Next Phase

Verify local connection states, checking that PostgreSQL migrations and Prisma definitions work. The next phase will establish User schemas, JWT Auth handlers, and role middlewares.